Design Pattern Factory

SEW





Zweck Factory

- Die **Factory** ist ein Erzeugungsmuster (Creational Design Pattern).
- Sie **kapselt die Objekterzeugung** in einer eigenen Methode.
- Der Client kennt nur das **Interface** des Produkts, nicht die konkrete Klasse.
- Unterklassen entscheiden, **welches** Objekt erzeugt wird.



Problem: new überall

Naheliegende Lösung: Der Client erzeugt die Objekte mit `new` und entscheidet per `if`/`switch` selbst.

Jede neue Variante (z. B. `Plane`) erfordert Änderungen **im Client-Code**.

→ Der Client ist fest an alle konkreten Klassen gekoppelt.

```
Transport t;
if (type == "road")
    t = new Truck();
else if (type == "sea")
    t = new Ship();
else if (type == "air")
    t = new Plane();
else
    throw new ArgumentException();
t.Deliver();
```

Dieser Block steht in jeder Stelle des Programms, die ein Transportmittel braucht.

Problem: Änderungen

Was passiert, wenn sich die Anforderungen ändern?

Enge Kopplung

Der Client kennt alle konkreten Klassen (Truck, Ship, Plane) und hängt von ihnen ab.

Duplizierter Code

Dieselbe if/else-Kette steht an vielen Stellen und muss überall angepasst werden.

Nicht erweiterbar

Neue Produkte erzwingen Änderungen an bestehendem Code (Open/Closed-Prinzip verletzt).

Ziel: Erzeugung und Verwendung trennen.

Open: Offen für Erweiterung

Closed: bestehender Code muss nicht verändert werden

Lösung: Factory-Muster

Die Objekterstellung wird in eine **eigene Methode** ausgelagert: die **Factory Method**.

Der Client bekommt ein **Interface** zurück und muss die konkrete Klasse nicht kennen: `CreateTransport()`.

Vorher

```
Transport t;
if (type == "road")
    t = new Truck();
else
    t = new Ship();
t.Deliver();
```



Nachher

```
Logistics logistics =
    new RoadLogistics();
ITransport t =
    logistics.CreateTransport();
t.Deliver();
```

Factory im Alltag: Pizza bestellen



Pizza bestellen



Bestellen



Rezept der Filiale



Backen



Fertig!

- **Du (Kund:in)** = Client: bestellst „eine Pizza“ und kennst nur das Interface
- **Pizzeria (allgemein)** = Creator: bietet die Methode „Pizza zubereiten“ an
- **Filiale Waidhofen / Neapel** = Concrete Creator: backt nach eigenem Rezept
- **Pizza** = Product: das fertige Ergebnis, egal welche Variante

Bestandteile

Product

Interface oder abstrakte Klasse für alle Objekte, die die Factory erzeugt.

Beispiel: ITransport

Concrete Product

Konkrete Implementierung des Produkts.

Beispiel: Truck, Ship

Creator

Deklariert die Factory Method, die ein Product zurückgibt. Enthält oft weitere Logik.

Beispiel: Logistics

Concrete Creator

Überschreibt die Factory Method und liefert ein konkretes Product.

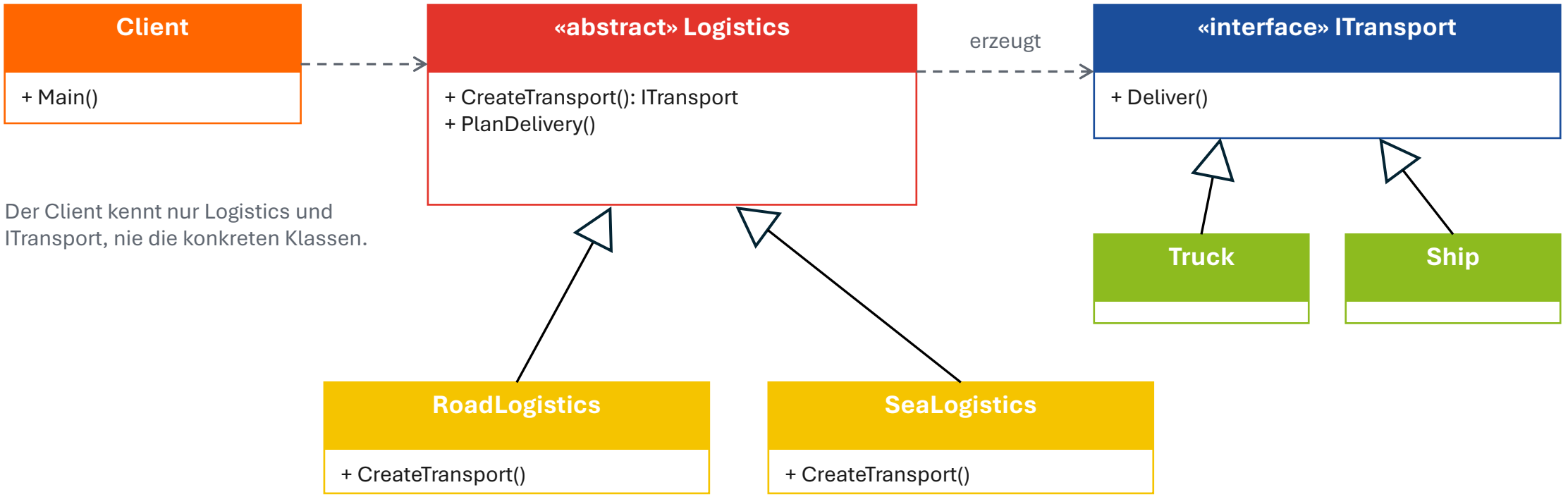
Beispiel: RoadLogistics

Client

Arbeitet nur mit Creator und Product (Interfaces).

Beispiel: Program.Main()

Struktur



Der Client kennt nur Logistics und ITransport, nie die konkreten Klassen.

Pfeil mit Dreieck = „erbt von / implementiert“

Die Factory-Familie

„Factory“ ist ein Sammelbegriff: Drei Varianten sollte man unterscheiden.

Simple Factory

Eine Klasse mit `static` Methode und `switch`. Kein offizielles GoF-Muster, aber sehr verbreitet.

Factory Method

Unterklassen überschreiben die Methode und entscheiden, was erzeugt wird. **Thema dieser Einheit.**

Abstract Factory

Erzeugt ganze **Familien** zusammengehöriger Objekte (z. B. Button + Checkbox pro Design).

Beispiel: Notification

INotification.cs

Product

EmailNotification.cs
SmsNotification.cs

Concrete Products

NotificationCreator.cs

Creator

EmailCreator.cs
SmsCreator.cs

Concrete Creators

Program.cs

Client

Ein Versandsystem soll Benachrichtigungen per E-Mail oder SMS verschicken. Der Client soll nicht wissen, welche Klasse dafür zuständig ist.

Product

```
public interface INotification
{
    void Send(string message);
}

public class EmailNotification : INotification
{
    public void Send(string message) => Console.WriteLine($"E-Mail: {message}");
}

public class SmsNotification : INotification
{
    public void Send(string message) => Console.WriteLine($"SMS: {message}");
}
```

Product: gemeinsames Interface. Der Client verwendet nur `INotification`.

Creator

```
public abstract class NotificationCreator
{
    // Factory Method
    public abstract INotification CreateNotification();

    public void Notify(string message)
    {
        INotification n = CreateNotification();
        n.Send(message);
    }
}
```

Factory Method

`abstract`: Unterklassen müssen sie umsetzen.

Notify()

Nutzt die Factory Method, ohne die konkrete Klasse zu kennen.

Concrete Creator

```
public class EmailCreator : NotificationCreator
{
    public override INotification CreateNotification()
    {
        return new EmailNotification();
    }
}

public class SmsCreator : NotificationCreator
{
    public override INotification CreateNotification()
    {
        return new SmsNotification();
    }
}
```

Concrete Creator

Jeder Creator liefert genau ein Product.

Neue Variante = neue Klasse. Bestehender Code bleibt unverändert.

Client

```
// 1) Der Client arbeitet nur mit dem abstrakten Creator
NotificationCreator creator = new EmailCreator();
creator.Notify("Willkommen!");      // E-Mail: Willkommen!

// 2) Anderer Creator -> anderes Produkt, gleicher Client-Code
creator = new SmsCreator();
creator.Notify("Dein Code: 4711");  // SMS: Dein Code: 4711

// 3) Auch das Produkt allein ist nutzbar
INotification n = new SmsCreator().CreateNotification();
n.Send("Hallo");
```

- Welcher Creator verwendet wird, entscheidet z. B. die Konfiguration, nicht der restliche Code.

Zwei Wege zur Factory

Factory Method (Vererbung)

- Eine Unterklasse pro Produkt.
- Ideal, wenn die Varianten stabil sind.
- Erweiterung ohne Änderung bestehenden Codes.

Simple Factory (Parameter)

- Eine Methode mit switch.
- Ideal für kleine, überschaubare Fälle.
- Neue Variante ändert die Factory.

```
public static INotification Create(string type) => type switch
{
    "email" => new EmailNotification(),
    "sms" => new SmsNotification(),
    _ => throw new ArgumentException(type)
};
```

Factory im .NET-Alltag

IEnumerable

```
IEnumerator<int> e = list.GetEnumerator();
// jede Collection erzeugt ihren eigenen Enumerator
```

ILoggerFactory

```
ILogger logger = loggerFactory.CreateLogger("App");
// Logger-Typ wird von der Factory bestimmt
```

IHttpClientFactory

```
HttpClient client = factory.CreateClient();
// ASP.NET Core: verwaltet Lebensdauer und Konfiguration
```

Vorteile und Nachteile

Vorteile

+ **Lose Kopplung**: Der Client kennt nur Interfaces.
+ **Open/Closed**: Neue Produkte ohne Änderung bestehenden Codes.
+ Erzeugungscode steht **an einer Stelle**.
+ **Gut testbar**: Creator lassen sich durch Test-Varianten ersetzen.

Nachteile

− Viele **zusätzliche Klassen** (pro Produkt oft ein Creator).

− Für einfache Fälle meist übertrieben.

Factory vs. Builder

Factory

- Entscheidet, welcher Typ erzeugt wird.
- Ein Aufruf liefert das fertige Objekt.
- Fokus: Welche Klasse?

Builder

- Baut ein komplexes Objekt schrittweise auf.
- Mehrere Aufrufe, am Ende Build().
- Fokus: Wie wird das Objekt aufgebaut?

Beide Muster lassen sich kombinieren: Eine Factory kann intern einen Builder verwenden.

Übung: Logistik-Factory

Setzt das Factory-Muster für ein **Logistiksystem** um (siehe Aufgabe_Factory.pdf).

Product

`ITransport` mit `Deliver()` und `Name`

Concrete Products

`Truck`, `Ship` und `Plane` mit eigener Ausgabe

Creator

`Logistics` mit `CreateTransport()` und `PlanDelivery()`

Concrete Creators

`RoadLogistics`, `SeaLogistics`, `AirLogistics`

Client

Alle drei Logistics-Varianten verwenden, Lieferung planen und ausgeben